

Secure Multiplayer Lobby Protocol

Autor: Konrad Iwanczewski

Github: <https://github.com/AtelierRq/Secure-Multiplayer-Lobby-Protocol>

Etap 1 — Specyfikacja autorskiego protokołu sieciowego

1. Cel protokołu i zakres

a) Do czego służy protokół?

SMLP (Secure Multiplayer Lobby Protocol) to protokół aplikacyjny przeznaczony do obsługi komunikacji klient–serwer w systemie lobby multiplayer.

Protokół umożliwia:

- logowanie użytkowników,
- tworzenie lobby,
- dołączanie do lobby,
- wymianę wiadomości między użytkownikami,
- synchronizację stanu graczy,
- utrzymywanie aktywnej sesji,
- rozpoczęcie sesji gry.

b) Jakie problemy rozwiązuje?

Protokół rozwiązuje problem organizacji i synchronizacji komunikacji pomiędzy wieloma klientami w środowisku multiplayer.

SMLP zapewnia:

- bezpieczną komunikację,
- autoryzację użytkowników,
- zarządzanie lobby,
- kontrolę stanów użytkowników,
- niezawodną wymianę komunikatów,
- obsługę błędów połączenia.

c) W jakim modelu działa?

Protokół działa w modelu klient–serwer.

Serwer:

- utrzymuje listę aktywnych klientów,
- przechowuje informacje o lobby,
- synchronizuje stan użytkowników,
- odpowiada za autoryzację,
- przekazuje komunikaty pomiędzy klientami.

Klient:

- wysyła żądania do serwera,
- odbiera komunikaty i aktualizacje stanu,
- wykonuje operacje związane z lobby.

2. Założenia techniczne

a) Transport

SMLP wykorzystuje:

- TCP jako warstwę transportową,
- TLS 1.2 jako mechanizm szyfrowania komunikacji.

TCP zapewnia:

- niezawodność transmisji,
- zachowanie kolejności pakietów,
- retransmisję utraconych danych.

TLS zapewnia:

- poufność danych,
- integralność komunikacji,
- uwierzytelnienie serwera.

b) Kodowanie wiadomości

Komunikaty są kodowane tekstowo w autorskim formacie opartym o separator „|”.

Ogólny format:

TYPE|FIELD1|FIELD2|...\n

Każdy komunikat:

- kończy się znakiem nowej linii,
- posiada typ komunikatu,
- może zawierać dodatkowe pola.

c) Założenia dotyczące niezawodności

Warstwa TCP odpowiada za:

- retransmisję,
- kolejność pakietów,
- kontrolę błędów transmisji.

Warstwa aplikacyjna odpowiada za:

- timeout sesji,

- heartbeat,
- walidację komunikatów,
- obsługę błędnych stanów,
- wykrywanie nieaktywnych klientów.

3. Struktura komunikatów

a) Typy wiadomości

Autoryzacja

TYP	OPIS
LOGIN	logowanie użytkownika
LOGIN_OK	poprawne logowanie
LOGIN_FAIL	nieudane logowanie
TOKEN	token sesyjny
LOGOUT	wylogowanie

Lobby

TYP	OPIS
CREATE_LOBBY	utworzenie lobby
LOBBY_CREATED	potwierdzenie utworzenia
JOIN	dołączenie do lobby
JOIN_OK	poprawne dołączenie
LEAVE	opuszczenie lobby
PLAYER_JOINED	nowy gracz
PLAYER_LEFT	gracz opuścił lobby
LIST_LOBBIES	lista lobby

Gra i synchronizacja

TYP	OPIS
READY	gotowość gracza
NOT_READY	anulowanie gotowości
START	rozpoczęcie gry

GAME_START | rozpoczęcie sesji

Komunikacja

TYP	OPIS
CHAT	wiadomość tekstowa
CHAT_BROADCAST	wiadomość do lobby
PING	heartbeat
PONG	odpowiedź heartbeat
ACK	potwierdzenie
ERROR	komunikat błędu
BYE	zakończenie połączenia

b) Pola wiadomości

LOGIN

LOGIN|username

CREATE_LOBBY

CREATE_LOBBY|room_name|max_players

JOIN

JOIN|room_id

CHAT

CHAT|message

ERROR

ERROR|error_code|description

c) Format przykładowej ramki

Poprawna wiadomość:

CHAT|Hello everyone

Niepoprawna wiadomość:

CHAT

Brak wymaganego pola „message”.

d) Walidacja komunikatów

Serwer sprawdza:

- poprawność typu komunikatu,
- liczbę pól,
- długość wiadomości,
- aktualny stan klienta,
- poprawność tokenu sesyjnego.

Niepoprawne komunikaty powodują wystanie:

ERROR|400|Bad message format

4. Model stanów i przebieg komunikacji

Role użytkowników

ROLA	OPIS
HOST	właściciel lobby, może rozpocząć sesję i zarządzać lobby
PLAYER	standardowy uczestnik lobby

a) Stany klienta

STAN	OPIS
DISCONNECTED	brak połączenia
CONNECTED	połączenie ustanowione
AUTH_PENDING	oczekiwanie na logowanie
AUTHENTICATED	użytkownik zalogowany
IN_LOBBY	użytkownik w lobby
READY	użytkownik gotowy
IN_GAME	rozpoczęta sesja

b) Przebieg komunikacji

1. Nawiązanie połączenia

Klient ustanawia połączenie TCP i TLS.

2. Logowanie

Klient wysyła:

LOGIN|player1

Serwer odpowiada:

LOGIN_OK|session_token

3. Tworzenie lobby

Klient:

CREATE_LOBBY|Arena|4

Serwer:

LOBBY_CREATED|101

4. Dołączenie graczy

Nowy klient:

JOIN|101

Serwer:

JOIN_OK|101

5. Gotowość

Klienci:

READY

6. Start sesji multiplayer

Po otrzymaniu stanu READY od wszystkich graczy host może rozpocząć sesję.

Host:

START

Serwer:

GAME_START

c) Timeouty i retransmisja

Heartbeat

Co 10 sekund klient wysyła:

PING

Serwer odpowiada:

PONG

Timeout

Brak heartbeat przez 30 sekund powoduje:

- rozłączenie klienta,
- usunięcie z lobby.

5. Bezpieczeństwo

a) Realizacja bezpieczeństwa komunikacji

I. Poufność

Poufność zapewnia TLS 1.2.

Wszystkie dane aplikacyjne są szyfrowane.

II. Integralność

Integralność komunikacji zapewnia:

- TLS,
- opcjonalny HMAC wiadomości.

III. Uwierzytelnienie

Serwer posiada certyfikat TLS.

Klient weryfikuje certyfikat serwera przy użyciu własnego CA.

IV. Autoryzacja

Po poprawnym logowaniu klient otrzymuje token sesyjny.

SMLP wykorzystuje uproszczony model logowania oparty wyłącznie o unikalny nickname użytkownika.

Token może być wykorzystywany do:

- autoryzacji operacji,
- wznowienia sesji.

V. Ochrona przed replay attack

Każdy komunikat może zawierać:

- timestamp,
- request ID.

Serwer odrzuca stare lub zduplikowane komunikaty.

VI. Krótki model zagrożeń

ZAGROŻENIE	MECHANIZM OCHRONY
PODSŁUCH	TLS
KOMUNIKACJI	
MODYFIKACJA	TLS / HMAC
DANYCH	
NIEAUTORYZOWANY	logowanie + token
DOSTĘP	
REPLAY ATTACK	timestamp / request ID
FLOOD	rate limiting
KOMUNIKATAMI	

6. Obsługa błędów i awarii połączenia

a) Kody błędów

KOD	ZNACZENIE
400	błędny format wiadomości
401	unauthorized
403	forbidden
404	lobby not found

408	timeout
409	nickname already used
429	rate limit exceeded
500	internal server error

b) Zachowanie po błędach składniowych

Niepoprawny komunikat powoduje:

- wysłanie ERROR,
- zapis do logów,
- możliwość rozłączenia klienta po wielu błędach.

c) Timeout połączenia

Klient nieaktywny przez ponad 30 sekund:

- zostaje rozłączony,
- usunięty z lobby.

d) Utrata połączenia

Po utracie połączenia:

- serwer usuwa klienta,
- informuje pozostałych graczy.

e) Duplikaty wiadomości

Duplikaty mogą być wykrywane na podstawie:

- request ID,
- timestamp.

f) Limity i ochrona przed nadużyciami

Limity:

- maksymalna długość wiadomości: 1024 bajty,
- maksymalna liczba klientów w lobby,
- rate limiting komunikatów.

7. Przykładowe scenariusze komunikacji

Scenariusz 1 — poprawna sesja

Klient:

LOGIN|player1

Serwer:

LOGIN_OK|TOKEN123

Klient:

CREATE_LOBBY|Arena|4

Serwer:

LOBBY_CREATED|101

Scenariusz 2 — błędny format komunikatu

Klient:

JOIN

Serwer:

ERROR|400|Bad message format

Scenariusz 3 — nieautoryzowany użytkownik

Klient:

CREATE_LOBBY|Arena|4

Serwer:

ERROR|401|Unauthorized

Scenariusz 4 — timeout klienta

Klient nie wysyła heartbeat.

Serwer:

ERROR|408|Connection timeout

Następnie serwer zamyka połączenie.

Zarządzanie lobby

Host może:

- rozpocząć sesję multiplayer,
- usunąć lobby,
- zarządzać uczestnikami lobby.

Lobby jest automatycznie usuwane, gdy:

- wszyscy gracze opuszczą lobby,
- host zakończy sesję,
- nastąpi timeout wszystkich uczestników.

Podsumowanie

SMLP jest protokołem aplikacyjnym przeznaczonym do bezpiecznej komunikacji klient-serwer w systemie lobby multiplayer.

Protokół zapewnia:

- szyfrowanie TLS,
- obsługę stanów klienta,
- autoryzację,
- zarządzanie lobby,
- niezawodną komunikację,
- mechanizmy bezpieczeństwa,
- obsługę błędów i timeoutów.